# Lab 02: Algoritmos de Busca na Prática

## Implementando Dijkstra e A* em Python

Engenharia de Computação / Análise e Desenvolvimento de Sistemas
joaovasconcelos@univicosa.com.br

ADS308

# Objetivos do Laboratório

Hoje vamos traduzir a teoria em código funcional:

- **Modelagem:** Representar um mapa com obstáculos usando Matrizes (`numpy`).
- **Dijkstra na Prática:** Implementar a busca pelo caminho mais curto baseado apenas no custo real.
- **A* na Prática:** Adicionar a "inteligência"da Heurística.
- **Visualização:** Comparar visualmente o número de nós explorados por cada algoritmo.

# 1. Modelando o Problema (O Mapa)

Em Ciência de Dados, representamos espaços físicos como arrays 2D.

- 0 = Caminho livre
- 1 = Obstáculo (Parede)

## Criando a Grade com NumPy

```
import numpy as np

mapa = np.array([
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 0, 0, 0],
    [0, 0, 0, 0, 0]
])
inicio = (0, 0)
objetivo = (4, 4)
```

# 2. A Matemática da Heurística (A*)

Como só podemos andar na vertical e horizontal na nossa grade, a melhor heurística é a **Distância de Manhattan**.
A fórmula é: $h(n) = |x_1 - x_2| + |y_1 - y_2|$

## Implementação em Python

```
def heuristica(atual, objetivo):
    x1, y1 = atual
    x2, y2 = objetivo
    return abs(x1 - x2) + abs(y1 - y2)
```

*Nota: No algoritmo de Dijkstra, nós simplesmente ignoramos essa função!*

# 3. O Motor: Fila de Prioridade

Ambos os algoritmos precisam escolher o próximo nó mais promissor. No Python, usamos a biblioteca `heapq` para criar uma fila de prioridade de forma eficiente.

## Exemplo de uso do heapq

```
import heapq

fila = []
# O primeiro valor da tupla é a prioridade (ex: f_score)
heapq.heappush(fila, (5, "Nó C"))
heapq.heappush(fila, (1, "Nó A"))
heapq.heappush(fila, (3, "Nó B"))

# heappop sempre tira o de menor valor primeiro!
menor = heapq.heappop(fila) # Retorna (1, "Nó A")
```

# 4. Implementando o Dijkstra

Lembre-se: O Dijkstra usa apenas o custo real $g(n)$.

## O Coração do Dijkstra ($f = g$)

```
# Fila guarda (g_score, (x,y))
heapq.heappush(fila, (0, inicio))
g_score = {inicio: 0}

while fila:
    custo_atual, atual = heapq.heappop(fila)
    if atual == objetivo: break # Achou!

    for vizinho in vizinhos_validos(atual, mapa):
        novo_g = g_score[atual] + 1 # Custo do passo é 1

        if vizinho not in g_score or novo_g < g_score[vizinho]:
            g_score[vizinho] = novo_g
            heapq.heappush(fila, (novo_g, vizinho))
```

# 5. Implementando o A*

O A* combina o custo real com a heurística: $f(n) = g(n) + h(n)$.

## O Coração do A* ($f = g + h$)

```
# Fila guarda (f_score, (x,y))
heapq.heappush(fila, (heuristica(inicio, objetivo), inicio))
g_score = {inicio: 0}

while fila:
    f_atual, atual = heapq.heappop(fila)
    if atual == objetivo: break # Achou!

    for vizinho in vizinhos_validos(atual, mapa):
        novo_g = g_score[atual] + 1

        if vizinho not in g_score or novo_g < g_score[vizinho]:
            g_score[vizinho] = novo_g
            f_score = novo_g + heuristica(vizinho, objetivo)
```

# 6. O Poder da Visualização

No Jupyter Notebook que vocês receberão, utilizaremos o `matplotlib` para pintar na tela:

- **Preto:** Paredes/Obstáculos.
- **Vermelho:** O caminho final encontrado.
- **Azul Claro:** Todos os nós que o algoritmo "visitou"(*explorou*) para tentar achar o caminho.

*É aqui que vocês verão a grande diferença entre o algoritmo que anda às cegas (Dijkstra) e o algoritmo com bússola (A*).*

## Hora de Codar!

O código base incompleto está no Google Colab e deverão:

1. Fazer uma cópia para o Google Colab pessoal
2. Preencher a lógica da Distância de Manhattan.
3. Completar a inserção na fila do Dijkstra e do A*.
4. **Análise Crítica:** Anotar no Notebook quantos nós o Dijkstra precisou explorar para achar a saída contra quantos o A* explorou no mesmo mapa.
5. Modificar o labirinto para forçar o A* a entrar em um "beco sem saída" e analisar o comportamento.